# Q-PROVE: HMAC-based Provenance and Validity Verification for QR-Encoded URLs*

KyungA Kang[1], Soowang Lee[2] and Jiyoon Kim[1†]
[1] Dept. of Computer Science and Engineering, Gyeongsang National University, Republic of Korea
{kka2418, jykim92}@gnu.ac.kr
[2] Dept. of AI Convergence Engineering, Gyeongsang National University, Republic of Korea
id0311sw@gnu.ac.kr

**Abstract**

A growing number of attacks have shown that Quick Response (QR) codes are being abused as an effective phishing (quishing) channel that bypasses traditional email and web defenses. Existing countermeasures mainly rely on destination-URL classification, cryptographic watermarking, or specialized secure scanners, and therefore either remain vulnerable to evolving infrastructure or require non-standard encoders and decoders. In this paper, a QR-centric integrity and provenance framework is proposed that strengthens trust in QR-encoded URLs at both generation and scan time. Before a QR code is created, the target URL is normalized into a canonical form, augmented with issuer and validity metadata, and protected using a Hash-based Message Authentication Code (HMAC); the resulting authenticated payload is then encoded into a standard QR symbol. When the code is scanned, the client verifies the HMAC and validity interval locally and informs the user about the issuer and trust status before following the URL. This design allows counterfeit, tampered, or expired codes to be rejected without online lookups while remaining compatible with existing QR decoding infrastructure, and it complements detection-based and visual anti-tampering approaches in a layered defense against quishing.

* Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 17, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).
† Corresponding author

# 1 Introduction

Quick Response (QR) codes have been widely deployed as a lightweight interface for payments, authentication, logistics, and public services, building on their high data density and ease of use with commodity smartphones [1]. QR code–based payment services have particularly expanded in both developed and emerging markets, and the global QR code payment market has been projected to grow rapidly over the next decade, indicating that QR-mediated interactions will remain a central channel for digital transactions [2]. As QR codes increasingly serve as the first touchpoint of end-to-end business processes rather than as auxiliary identifiers, the security and reliability of QR-driven workflows have been recognized as a key prerequisite for trustworthy digital ecosystems [3].

In parallel with this diffusion, a line of research has been developed that systematically examines the security and privacy implications of QR code usage across applications and platforms. Comprehensive surveys have been conducted to catalog prominent attack vectors, such as payment fraud, code counterfeiting, information leakage, and reader-application abuse, and to map corresponding response techniques from both the academic and industrial domain. In addition, the usability and deployability of cryptographic protection mechanisms, such as Elliptic Curve Digital Signature Algorithm (ECDSA), Hash-based Message Authentication Codes (HMAC), and Advanced Encryption Standard (AES) embedded into QR codes, have been explored to understand how authenticity and integrity guarantees can be balanced with constraints on code size, print quality, and user experience [4].

Within this broader landscape, QR code phishing (Quishing) has been identified as one of the most pressing emerging threats linked to real-world QR deployments. Field studies and naturalistic experiments have been carried out to observe how users interact with malicious QR codes embedded in plausible scenarios, and to characterize behavioral patterns and contextual factors that increase susceptibility to QR-mediated phishing [5-6]. In parallel, organizational measurements of phishing campaigns have begun to include QR-based lures alongside traditional hyperlink-based emails, highlighting that quishing is being operationalized as part of mainstream social-engineering toolkits rather than remaining an isolated niche [7]. Taken together, these works indicate that the human-factor dimension of QR phishing, ranging from trust in physical signage to habits around mobile scanning, is becoming an active subtopic in usable security and phishing research.

At the system level, attention has also shifted from viewing QR codes solely as carriers of URLs toward analyzing the complete processing pipeline involving in-app scanners, operating-system components, and backend services [3]. Studies of in-app QR code scanning have revealed that decoded content is frequently passed directly to internal handlers, WebView components, or privileged functions, thereby exposing new avenues for remote invocation of sensitive operations when crafted QR payloads are presented to vulnerable applications [8]. [9] have argued that QR security must be considered as a multi-layer design problem, in which physical media, mobile applications, and server-side logic form a combined attack surface rather than independent components. These perspectives suggest that effective defenses against QR phishing cannot be confined to user education or simple URL filtering alone, but must account for the behavior of the surrounding software and infrastructure.

In response to these challenges, several technical directions have been pursued for strengthening QR security, each with distinct assumptions and limitations. [3] and [4] have focused on embedding cryptographic authenticity and integrity into QR codes so that tampering and forgery can be detected by compliant scanners, while [10] has investigated secure scanner designs that integrate policy checks and permission controls into the decoding path. [11] has explored machine-learning-based detection of malicious Uniform Resource Locators (URLs) and related features derived from QR content, demonstrating that real-time classification of benign and malicious codes is feasible on commodity hardware. However, [12] have emphasized that these approaches only partially address the

characteristics of modern quishing campaigns, which often exploit cross-device workflows, rapidly changing infrastructure, and contextual cues that are not fully captured by generic URL-centric models.

Given this evolution of threats and defenses, a need has been recognized for QR phishing detection methods that explicitly target quishing scenarios and their operational context, rather than treating QR codes simply as another URL encoding format. In this work, a QR phishing detection system is proposed that is designed to operate at scan time, to analyze both decoded content and QR-specific contextual features, and to provide early risk signals before authentication, payment, or other security-sensitive actions are completed. By aligning its threat model with the trends identified in prior studies on QR security, user behavior, and in-app vulnerabilities, the proposed system is intended to complement existing cryptographic and scanner-hardening techniques and to address a gap between the sophistication of current quishing attacks and the protection afforded by deployed defenses.

This paper is organized as follows. In Section 2, we present the background, including a literature review and a detailed discussion of QR code generation and usage processes as well as their vulnerabilities. In Section 3, we describe the proposed system for QR code phishing detection. Finally, Section 4 concludes the paper and outlines directions for future work.

# 2 Background

## 2.1 Literature Review

In recent years, the threat of QR code phishing, often referred to as quishing, has begun to be treated as a distinct research topic rather than a marginal variant of traditional hyperlink-based phishing [7, 13]. [13] has explicitly classified QR code–mediated attacks as a separate delivery medium and discussed them in the context of multi-channel social-engineering campaigns, emphasizing their ability to bypass conventional email and browser defenses.

At the organizational level, Weinz et al. [7] conducted large-scale phishing simulations across several companies, comparing classic click-through emails, QR code phishing emails, and messages written with the assistance of large language models (LLMs). Their results indicated that quishing emails were at least as effective as traditional phishing emails at driving users to landing pages, while being harder for operational detectors to flag, thus highlighting a structural blind spot in current enterprise defenses.

Building on this threat characterization, multiple technical directions have been explored for detecting and mitigating quishing attacks using machine learning (ML) and artificial intelligence (AI). [11] proposed an ML-based detector that jointly analyzes phishing links and QR codes in Unified Payments Interface (UPI) transactions, demonstrating that lexical features of embedded URLs can be used to flag fraudulent payment requests in real time. In a supplementary study on QR code infrastructure security [15], an AI-based system combining anomaly detection, risk signals, and multiple machine learning classifiers was used to identify QR-based phishing, data leakage, and injection attempts at the infrastructure level.

A different stream of work has examined quishing-specific defenses that move beyond simple URL analysis. [14] introduced self-authenticating dual-modulated QR (SDMQR) codes, which embed cryptographic signatures into visually compatible QR patterns so that scanners can verify whether a code originates from a pre-registered, trusted issuer before following the embedded link. This approach has been positioned as a structural countermeasure against code replacement and tampering in both physical and digital environments, with the explicit goal of quishing attacks by preventing users from being silently redirected to attacker-controlled destinations.

Recent works have also contributed QR-centric verification mechanisms that, while not limited to phishing, can be leveraged to strengthen defenses against quishing. A deep learning–based verification

scheme [16] proposed to watermark QR codes with hidden security features and to use Siamese neural networks to distinguish genuine codes from forgeries, achieving high accuracy in detecting copied or tampered codes. Further, [17] presented robust QR code extraction and verification techniques tailored to noisy, real-world environments, improving the reliability of downstream authenticity checks.

Collectively, these studies indicate that current research on quishing and QR-centric attacks is converging on three main directions: empirical measurement of attack effectiveness, cryptographic and protocol-level hardening of QR codes, and ML-based detection frameworks that treat the QR symbol and its context as first-class security objects.

## 2.2 Overview of QR Code

Quick Response (QR) codes are two-dimensional matrix barcodes that encode information in an arrangement of black and white modules, typically within a square symbol that can be captured by commodity camera sensors and decoded in real time. In contrast to one-dimensional barcodes that represent data along a single axis, QR codes support multiple standardized encoding modes—such as numeric, alphanumeric, and byte data—and incorporate Reed–Solomon–based error correction, which allows partial recovery of the payload even when portions of the symbol are occluded or degraded [15-16]. Typical QR layouts include finder patterns in three corners, alignment patterns for higher versions, timing patterns, and format and version information fields, all of which are used by decoders to locate the code, correct geometric distortions, and determine the encoding parameters.

From a systems perspective, QR codes function as a compact carrier of application-specific tokens that are interpreted by the scanning device and associated software. In industrial and robotics scenarios, for example, each QR symbol may encode global coordinates and identifiers that are used as artificial landmarks to correct localization error during indoor navigation [18]. In authentication and access-control contexts, QR codes are used to encapsulate credential data, one-time tokens, or references to server-side state, and [13] have discussed the integration of QR-encoded secrets with biometric authentication and cryptographic protection mechanisms. Across these domains, the decoding process generally consists of image acquisition, binarization, symbol detection, geometric normalization, Reed–Solomon decoding, and application-specific parsing of the recovered payload [16].

This generic pipeline has motivated a growing body of work on robust QR extraction and verification under real-world conditions. Deep learning–based methods [16] have been proposed to improve the detection and segmentation of QR regions in images with complex backgrounds, variable lighting, and physical distortions, thereby increasing the reliability of downstream decoding. [15] and [16] have shown that convolutional neural networks and lightweight architectures such as ShuffleNet can be trained to discriminate authentic codes from sophisticated counterfeits, even when the forgeries are printed and re-captured, which is critical for high-stakes applications such as product traceability and ticketing. As a result, modern QR ecosystems are increasingly characterized not only by generic encoding and decoding standards, but also by application-specific verification layers that seek to ensure that a decoded symbol is both syntactically valid and semantically trustworthy.

## 2.3 Vulnerability of QR Code

Beyond their functional advantages, QR codes exhibit several inherent vulnerabilities that arise from the opacity of their content and their integration into end-to-end service workflows. QR codes span the physical media on which codes are printed, the scanner and application that interpret them, and the backend systems that process the resulting inputs. A key observation is that users can only visually inspect a pattern of modules, while the actual target of the service is the decoded data, which may trigger browser navigation, in-app screen transitions, token transfers, or transaction requests within a very short

time frame after scanning. Consequently, QR-related vulnerabilities must be understood as a multi-stage problem involving generation, distribution, scanning context, and subsequent processing.

The Vulnerability of QR codes can be categorized into four main categories: social-engineering-based inducement threats, application and reader abuse threats, physical tampering and cloning threats, and input-abuse threats. Social-engineering-based inducement threats are characterized by attackers crafting emails, notices, or posters that resemble legitimate procedures—such as account reactivation, payment confirmation, or discount offers—in order to persuade users to scan embedded QR codes without suspicion; the primary consequences include redirection to phishing sites and credential theft [7]. Application and reader abuse threats exploit the fact that many mobile applications integrate in-app scanners that automatically connect decoded content to internal functions, so that a malicious QR code can effectively serve as a remote input to invoke sensitive operations, leak tokens, or access internal data if parsing and handler logic are not carefully constrained [8].

Physical tampering and cloning threats arise from the dual nature of QR codes as both digital identifiers and physical artifacts attached to surfaces in the environment [5]. In this category, attackers may overlay malicious stickers on top of legitimate codes, transplant copied codes to different locations, or reprint altered codes to change payment destinations or service endpoints, thereby manipulating the object that users believe they are scanning. Because the attack targets the physical carrier rather than the decoding algorithm, purely URL-based checks are insufficient to detect such substitutions. Finally, input-abuse threats occur when decoded QR payloads are forwarded directly into backend systems—such as inventory management, access control, logistics, or control interfaces—without adequate validation; in these scenarios, QR data effectively becomes untrusted external input that can contribute to Structured Query Language (SQL) injection, command injection, script execution, or unintended control-flow branching.

# 3 Proposed System

## 3.1 Preliminary

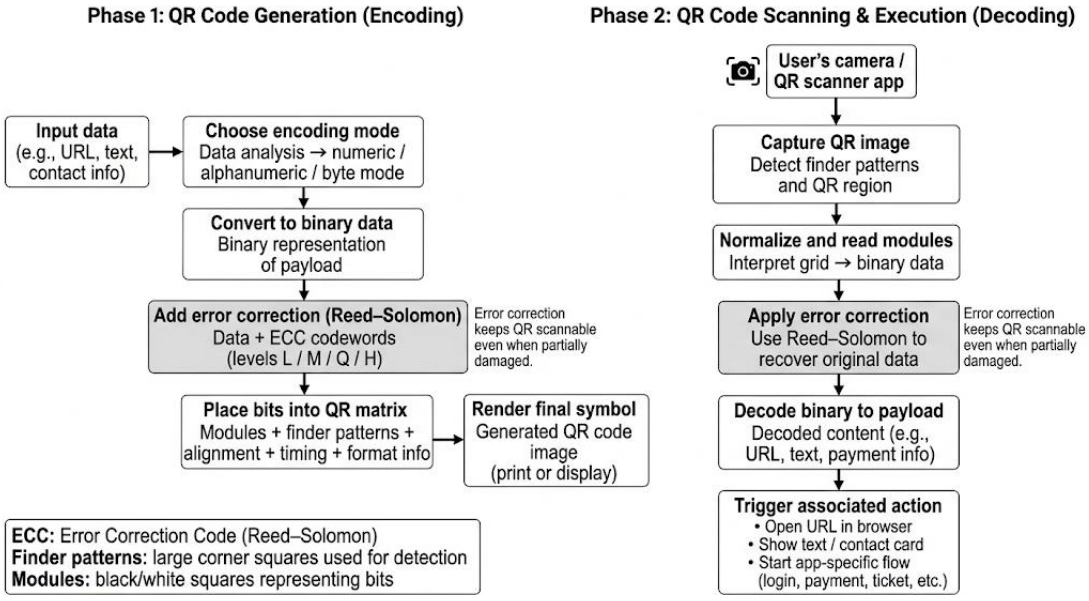



**Figure 1**. QR Code Generation and Scanning Procedure

In typical deployments, Quick Response (QR) codes are generated by taking an application-specific payload, most commonly a Uniform Resource Locator (URL), and encoding it into the two-dimensional matrix symbol defined by the QR standard. During generation, the payload is first converted into a sequence of bits according to one of the supported encoding modes (numeric, alphanumeric, byte, or kanji), after which Reed–Solomon error-correction codewords are added and the resulting bitstream is mapped onto the module grid while reserving positions for finder patterns, timing patterns, alignment patterns, and format and version information. The final image is printed on physical media or rendered on a display and is treated as an opaque token by users, who cannot visually infer the embedded payload or its security properties.

At scan time, the QR image is captured by a camera and processed by a decoding library that performs binarization, symbol localization, geometric normalization, and Reed–Solomon decoding to reconstruct the original payload bits. The decoded string is then passed to the surrounding application logic, which interprets it as a URL, a deep link, a payment request, or another type of instruction, and triggers follow-up actions such as opening a browser, switching to an in-app page, transmitting authentication tokens, or initiating an order or payment workflow. Because these follow-up actions are usually executed within a short time window and may involve privileged operations, the security of the overall QR interaction depends not only on the correctness of symbol decoding but also on the integrity, authenticity, and freshness of the underlying payload and on the way the application handles it.

## 3.2 Proposed System

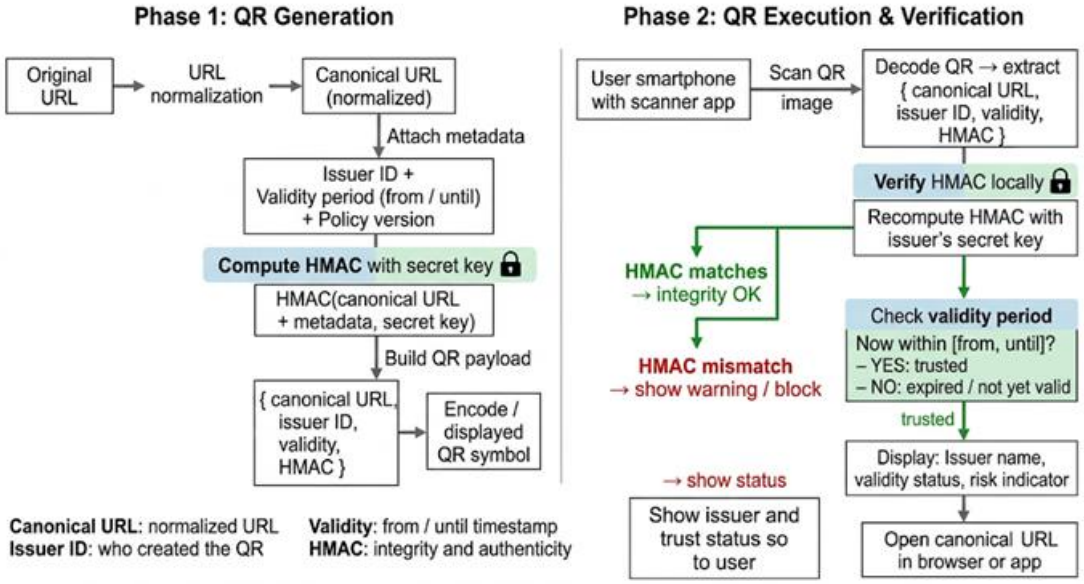



**Figure 2**. Proposed System

In the proposed system, the focus is placed on strengthening the integrity and provenance guarantees of QR-encoded URLs by introducing a normalization and keyed-authentication layer prior to QR generation. Before a QR code is created, the input URL is first normalized into a canonical representation that removes superficial variability, such as equivalent schemes, redundant path segments, or re-ordered query parameters, so that the same logical resource is always mapped to a unique, stable string. This canonical URL is then combined with issuer metadata (for example, an issuer identifier and policy version) and validity constraints (such as an expiration timestamp or validity interval) to form a structured payload that captures both the target resource and its contextual security attributes.

Over this structured payload, a Hash-based Message Authentication Code (HMAC) is computed using a secret key held by the trusted issuer or issuing backend, and the resulting authentication tag is bundled together with the canonical URL and metadata as the data field that is ultimately encoded into the QR symbol. As a result, each generated QR code contains not only the destination URL but also an integrity-protected record of who created the code and until when the code is to be considered trustworthy, all bound together under a single HMAC value that cannot be forged without access to the issuer's key. The overall generation process therefore produces QR codes that are self-describing with respect to issuer identity and validity period, while remaining compatible with standard QR decoding libraries.

When such a QR code is scanned, the client application does not immediately follow the embedded URL. Instead, the encoded payload is parsed into its constituent fields, canonical URL, issuer identifier, validity information, and HMAC tag, and a local verification step is performed. Using the pre-configured secret key (or a key derived from an issuer-specific master key), the client recomputes the HMAC over the received canonical URL and metadata and compares the result with the embedded tag; if the values do not match, the code is treated as tampered or counterfeit and the user is warned or the interaction is blocked. In parallel, the validity interval is checked against the current time, and codes

whose validity window has expired or is not yet active are rejected or displayed as untrusted, thereby limiting the impact of replay and long-term reuse of leaked codes.

Only when both integrity and freshness checks succeed is the canonical URL handed over to the browser or application component for normal execution, and the user interface can additionally display human-readable information about the issuer (for example, the service name or organization) and the validity status derived from the authenticated metadata. This design allows users and relying applications to distinguish between QR codes legitimately issued by registered services and visually similar codes that have been copied, altered, or created by attackers, even when the latter embed syntactically valid URLs. queries.

# 4 Conclusion

This paper has addressed the problem of QR code phishing by proposing a QR-centric integrity and provenance framework that operates at the point of QR code generation and scanning. The overall threat landscape and recent research on quishing and QR code security were first reviewed, highlighting that existing approaches are largely organized around detection-based filters, cryptographic integrity mechanisms, and physical or visual anti-tampering schemes. Building on this analysis, a system architecture was introduced in which URLs are normalized into canonical form, augmented with issuer and validity metadata, and protected by a Hash-based Message Authentication Code (HMAC) before being encoded into QR symbols. This design enables scanning clients to verify the integrity, issuer identity, and temporal validity of a QR code locally prior to following the embedded URL, thereby strengthening the trustworthiness of QR-mediated interactions without requiring changes to the visual appearance of codes or online reputation lookups.

Compared with prior work, the proposed system offers several advantages in terms of deployment model and security properties. Detection-oriented solutions based on machine learning classifiers or URL analysis can provide high accuracy at scan time, but they typically treat QR codes as opaque carriers and focus on labeling destinations as benign or malicious, making them sensitive to concept drift and zero-day infrastructure. Cryptographic and integrity-preserving schemes, including visual watermarking and self-authenticating QR variants, embed stronger authenticity guarantees into the symbol itself but often require customized encoders and decoders or impose non-trivial constraints on code density and visual design. In contrast, the proposed HMAC-based framework is designed to be agnostic to the visual layout of the code and to rely only on standard QR encoding and decoding capabilities, while still providing verifiable information about who issued the code and whether its trust window is active. By normalizing URLs before protection and binding issuer and validity metadata into a single authenticated payload, the system also mitigates risks arising from semantically equivalent but syntactically different URLs and from long-term reuse of leaked or cloned codes.

There remain, however, several directions in which the proposed approach can be extended and refined. One promising avenue is the integration of the HMAC-based trust layer with existing detection mechanisms, for example by using issuer and validity information as features in machine-learning-based quishing classifiers or as policy inputs in enterprise access-control systems. Another important direction is the exploration of key-management and delegation models that allow multiple organizations or services to participate in a shared QR trust infrastructure while preserving scalability and revocation capabilities. In addition, user-centric studies are needed to evaluate how issuer and validity indicators should be presented at scan time so that users can make informed decisions without incurring excessive cognitive load, particularly in contexts where benign and malicious codes coexist in the same physical environment. Future work will therefore focus on implementing and empirically evaluating the proposed system in realistic deployment scenarios, on formally analyzing its

security guarantees under various adversarial models, and on developing usability guidelines for integrating QR provenance and validity signals into mainstream scanning applications.

# Acknowledgment

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) [grant number RS-2026-25473237] and the Gyeongsang National University Glocal College 30 Project in 2026.

# References

[1] S. Tiwari, “An Introduction to QR Code Technology,” in Proceedings of the 2016 International Conference on Information Technology (ICIT), IEEE, 2016, pp. 39–44.
[2] Grand View Research, “QR Code Payment Market Size, Share & Trends, 2025–2033,” Market Research Report, Grand View Research, San Francisco, CA, USA, 2024.
[3] H. A. M. Wahsheh and F. L. Luccio, “Security and Privacy of QR Code Applications: A Comprehensive Study, General Guidelines and Solutions,” Information, vol. 11, no. 4, p. 217, 2020.
[4] R. Focardi, F. L. Luccio, and H. A. Wahsheh, “Usable Security for QR Code,” Journal of Information Security and Applications, vol. 48, p. 102369, 2019.
[5] L. J. L. Bekavac, S. Mayer, and J. Strecker, “QR-Code Integrity by Design,” in Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, pp. 1-9, 2024.
[6] F. Sharevski, M. M. Mossano, M. F. Veit, G. Schiefer, and M. Volkamer, “Exploring Phishing Threats through QR Codes in Naturalistic Settings,” in Symposium on Usable Security and Privacy (USEC 2024), Internet Society, 2024.
[7] M. Weinz, N. Zannone, L. Allodi, and G. Apruzzese, “The Impact of Emerging Phishing Threats: Assessing Quishing and LLM-generated Phishing Emails against Organizations,” in Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIACCS), ACM, 2025.
[8] X. Han, Y. Zhang, X. Zhang, Z. Chen, M. Wang, Y. Zhang, S. Ma, Y. Yu, E. Bertino, and J. Li, “Medusa Attack: Exploring Security Hazards of In-App QR Code Scanning,” in Proceedings of the 32nd USENIX Security Symposium, USENIX Association, 2023, pp. 4607–4624.
[9] M. S. Al-Zahrani, H. A. M. Wahsheh, and F. W. Alsaade, “Secure Real-Time Artificial Intelligence System against Malicious QR Code Links,” Security and Communication Networks, vol. 2021, Article ID 5540670, 18 pages, 2021.
[10] A. S. Rafsanjani, N. B. Kamaruddin, H. M. Rusli, and M. Z. Dabbagh, “QsecR: Secure QR Code Scanner According to a Novel Malicious URL Detection Framework,” IEEE Access, vol. 11, pp. 92523–92539, 2023.
[11] G.R. Charan and K.D. Thilak,“Detection of Phishing Link and QR Code of UPI Transaction using Machine Learning,” in Proceedings of the 2023 3rd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA), IEEE, 2023.
[12] D. Njuguna and J. Ndia, “Quick Response Code Security Attacks and Countermeasures: A Systematic Literature Review,” Journal of Cyber Security, vol. 7, no. 1, pp. 1–20, 2025.
[13] C. Barron and G. Sharma, “Quashing Quishing Attacks Using Self-Authenticating Dual-Modulated QR Codes,” IEEE Security & Privacy, vol. 24, no. 1, pp. 3–13, 2025.
[14] L. Rivas, V. K. Singh, V. Khandelwal, and L. Watkins, “Securing QR Codes Infrastructure Using AI to Detect Malicious Activity,” in Proc. 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC), 2025.
[15] V. L. Cu, X. V. Truong, H. V. Tran, H. T. Le, and H. V. Nguyen, “Deep Learning Based-Approach for Quick Response Code Verification,” Applied Intelligence, vol. 53, pp. 22700–22714, 2023.
[16] H. V. Nguyen et al., “Deep Learning Approaches Based Robust QR Code Extraction and Verification for Industrial IoT,” in Lecture Notes on Data Engineering and Communications Technologies, vol. 230, Springer, 2024, pp. 244–255.
[17] R. Goenka, M. Chawla, and N. Tiwari, “A Comprehensive Survey of Phishing: Mediums, Intended Targets, Attack and Defence Techniques and a Novel Taxonomy,” International Journal of Information Security, 2023.
[18] S.-H. Bach and P. Khoi, “Application of QR Code for Localization and Navigation of Indoor Mobile Robot,” IEEE Access, vol. 11, pp. 28384–28390, 2023.